Área de Arquitectura y Tecnología de Computadores (ARCOS)

Universidad Carlos III de Madrid

SISTEMAS OPERATIVOS

Práctica 2. Programación de un intérprete de comandos

(Uc3mshell)

Grado en Ingeniería Informática
Grado en Matemáticas y Computación
Doble Grado en Ingeniería Informática y Administración de Empresas

Curso 2025/2026

Tabla de contenido

1. Enunciado de la práctica....2
1.1. Descripción de la práctica....2
2. Consideraciones....3
2.1. Comandos....3
2.2. Secuencia de comandos....4
2.3. Redirecciones....4
2.4. Background (segundo plano)....5
2.5. Caso especial....5
3. Procesamiento de comandos....7
4. Desarrollo del intérprete de comandos Uc3mshell....9
4.1. Comando interno: exit....9
4.2. Comando interno: mycalc....10
4.3. Comando externo: mycp....10
5. Código fuente de apoyo....11
5.1. Comprobador proporcionado....12
6. Compilación y ejecución....13
7. Entrega....13
7.1. Plazo de entrega....13
7.2. Procedimiento de entrega de las prácticas....13
7.3. Documentación a entregar....13
8. Normas....16
9. Anexos....17
9.1. man function....17
9.2. Modo background y foreground....17
9.3. Comandos externos....18
10. Bibliografía....19

1. Enunciado de la práctica

Esta práctica permite al alumno familiarizarse con los servicios de gestión de procesos que proporciona POSIX. Asimismo, se pretende que conozca el funcionamiento interno de un intérprete de comandos (shell) en UNIX/Linux. En resumen, una shell permite al usuario comunicarse con el kernel del sistema operativo mediante la ejecución de comandos, ya sean simples o encadenados.

Para la gestión de procesos pesados, se utilizarán las llamadas al sistema de POSIX relacionadas, tales como *fork, wait* o *exit*. Para la comunicación entre procesos, igualmente se utilizarán las llamadas al sistema *pipe, dup, close* o *signal*.

El alumno debe diseñar y codificar, en lenguaje C y en el sistema operativo UNIX/Linux, un programa que actúe como intérprete de comandos o shell. El programa debe seguir estrictamente las especificaciones y los requisitos contenidos en este documento.

1.1. Descripción de la práctica

Similar al intérprete de comandos bash de Linux, el programa a desarrollar se llama "Uc3mshell" y debe leer línea a línea un fichero de comandos recibido como argumento de entrada y, mediante un intérprete de comandos, ejecutarlos como si se tratara de un terminal nativo.

La sintaxis para ejecutar este programa es la siguiente:

```shell
./uc3mshell <fichero de entrada>
```

A continuación, se muestra un ejemplo del contenido de un **fichero de entrada**:

```shell
## Uc3mshell P2
ls -l
cat fichero | grep palabra

cat fichero > fichero2 &
#ls -l | sort < fichero1
ls -l | sort < fichero2
```

La primera línea (## Uc3mshell P2) es **obligatoria**, por lo que el programa debe comprobar su existencia para saber si el fichero es un script. Si no existe, el programa deberá finalizar con un código de error -1 y devolver un error mediante la función **perror**.

Posteriormente, el programa desarrollado debe ser capaz de ejecutar, uno por uno, los comandos listados en el fichero. El script puede contener líneas vacías y comentarios que comienzan con "#". En este caso, el uc3mshell debe ignorar estas líneas y continuar con la siguiente. **El programa debe ejecutar todos los comandos, aunque estos terminen con error (el fallo de un comando no implica un fallo del intérprete).** El intérprete debe finalizar:

1. Cuando se han completado todos los comandos del fichero.
2. Cuando se ha producido un error en las llamadas al sistema utilizadas en el código (fork, exec, pipe, dup, etc.).
3. Cuando la primera línea del fichero de entrada sea diferente a "## Uc3mshell P2".

2. Consideraciones

1. Una línea puede ser un comando con o sin argumentos (p. ej., "ls", "ls -l", "ls -l directorio").
2. Una línea puede ser una secuencia de comandos encadenados (mediante pipes), cada uno con sus respectivos flags y argumentos (p. ej., ls -l | wc).
3. Cada línea de comandos puede contener los siguientes caracteres estandarizados:
   i. " **>** ": redirección de salida.
   ii. " **<** ": redirección de entrada.
   iii. " **!>** ": redirección de error.
   iv. " **&** ": indica que el comando se ejecutará en segundo plano.
4. Una línea que inicie con "#" es un comentario. Por ejemplo "# Aquí se imprime el resultado".

2.1. Comandos

Por lo tanto, **se entiende por comando** a una secuencia de textos separados por espacios. El primer texto especifica el nombre del comando a ejecutar. Las restantes son los argumentos del comando invocado. Por ejemplo, en el comando "ls -l .", el nombre del comando es "ls" y los parámetros son "-l" y ".".

El nombre del comando se pasa como argumento 0 (*man execvp*). Cada comando se ejecuta como un proceso hijo directo del intérprete (*man 2 fork*). El valor de un comando es

su estado de terminación (*man 2 wait*). Si la ejecución falla, se notifica el error por la salida estándar de error mediante la función **perror**.

2.2. Secuencia de comandos

Una secuencia de dos o más comandos es una serie de comandos separados por el carácter " | " (*man 2 pipe*). La salida del primer comando se pasa automáticamente como entrada al siguiente. El programa principal espera a que el último comando termine antes de aceptar una nueva instrucción. El resultado de toda la secuencia será el del último comando ejecutado. Por ejemplo, en "ls -l | wc -l" hay dos comandos: la salida del comando *ls* se pasa como entrada al comando *wc* para contar el número de líneas.

2.3. Redirecciones

Una **redirección** indica que la entrada o salida de datos de un comando debe provenir de o ir a un fichero en lugar de la terminal. La entrada o la salida de un comando o secuencia de comandos puede ser redirigida añadiendo tras él la siguiente notación:

- comando **< fichero** Usa un **fichero** como **entrada estándar** abriéndolo para lectura. Si el fichero no existe, termina con error (*man 2 open*). Las redirecciones de **entrada** solo aplican al **primer** comando:

```shell
comando1 < fichero | comando2 | comando3 ...
```

- comando **> fichero** Usa un **fichero** como **salida estándar**. Si el fichero no existe, se crea. Si existe, se trunca (*man 2 open / man creat*). Las redirecciones de **salida** solo aplican al **último** comando:

```shell
comando1 | comando2 | ... | comandoN > fichero
```

- comando **!> fichero** Usa un **fichero** como **salida estándar de error**. Si el fichero no existe, se crea. Si existe, se trunca (*man 2 open / man creat*). Las redirecciones de **error** solo aplican al **último** comando:

```shell
comando1 | comando2 | ... | comandoN !> fichero
```

En caso de cualquier error durante las redirecciones, se notifica por la salida estándar de error (**perror**), se suspende la ejecución de esa línea y se continúa con la siguiente línea.

Es posible hacer múltiples redirecciones en un mismo comando o secuencia de comandos. Por ejemplo, asumiendo que "file.out" existe:

```shell
cat < file.out | tail -x10 > tail.out !> tail.err
```

1. El comando "cat" leerá el fichero "file.out" y enviará su salida al comando "tail".
2. El comando "tail" fallará ya que "-x" no es una flag válida. Los ficheros "tail.out" y "tail.err" se crearán; el primero estará vacío, mientras que el segundo contendrá el mensaje de error por defecto: tail: *invalid option -- 'x'*.

2.4. Background (segundo plano)

Un comando o secuencia terminada en " **&** " supone la ejecución asíncrona del mismo; es decir, se ejecuta en **segundo plano**, por lo cual el programa principal continúa con la ejecución del resto de los comandos. Se debe considerar:

1. El proceso padre debe imprimir mediante la salida estándar el *pid* del proceso hijo creado y ejecutándose en segundo plano (Ver Anexos).
   Obligatorio, la impresión debe ser (sin comentarios extras):

```shell
printf("%d",pid);
```

2. Se debe gestionar correctamente la finalización de los procesos hijos, ya que, de no hacerlo, supondrá una penalización en la nota final.

2.5. Caso especial

Un **caso especial** es el comando "**echo**", en el que se debe controlar adecuadamente el manejo de comillas si existieran. Por ejemplo, los siguientes comandos generan exactamente la misma salida:

```shell
echo "Hola mundo"
echo Hola mundo
```

```
# Salida esperada
Hola mundo
Hola mundo
```

El código proporcionado **debe modificarse** para cumplir con estas **condiciones**.

3. Procesamiento de comandos

Para el desarrollo de este laboratorio, es necesario que los alumnos procesen un fichero de comandos. El nombre de este fichero se pasa como argumento al programa. **Deben leer carácter a carácter hasta encontrar un salto de línea e insertar un '\0' al final de la cadena de caracteres.** Una vez implementado este paso, cada una de las líneas puede ser pasada como parámetro a la función ***procesar_linea*** (*incluida en el código inicial uc3mshell.c*), la cual proporciona la funcionalidad básica para que el alumno pueda proceder con la gestión de los procesos.

Variables que serán utilizadas por los estudiantes, incluidas en el código inicial:

- **char * argvv[]:** estructura equivalente a "argv" que almacena la línea de comandos a la hora de ejecutar un programa. En este caso, esta estructura almacenará la línea correspondiente a un comando, junto con sus argumentos.
- **char * filev[3]:** Array que indica si hay una redirección desde o hacia un fichero. Cada posición se corresponde con:
    - filev[0]: redirección de entrada (descriptor de fichero 0).
    - filev[1]: redirección de la salida estándar (descriptor de fichero 1).
    - filev[2]: redirección de la salida de error (descriptor de fichero 2).

En caso de haber redirecciones, el contenido de la posición será el puntero al nombre del fichero. En caso contrario, el valor de la posición será *NULL*.

- **int background:** si el comando debe ejecutarse en segundo plano (background), el valor es 1; en otro caso, es 0.

El código inicial (*uc3mshell.c*) incluye, dentro de la función *procesar_linea*, un código que muestra por pantalla el contenido de argvv, de background y de redirecciones (filev). Se recomienda evaluar distintos casos una vez que se ha realizado la lectura y el entendimiento del fichero de comandos. Posteriormente, se recomienda modificar ese código para agregar la funcionalidad de ejecución de procesos.

EJEMPLO: Si el fichero de entrada contiene el siguiente comando:

```shell
ls -l | sort < fichero
```

Los argumentos anteriores tendrán la disposición que se muestra en la siguiente figura:

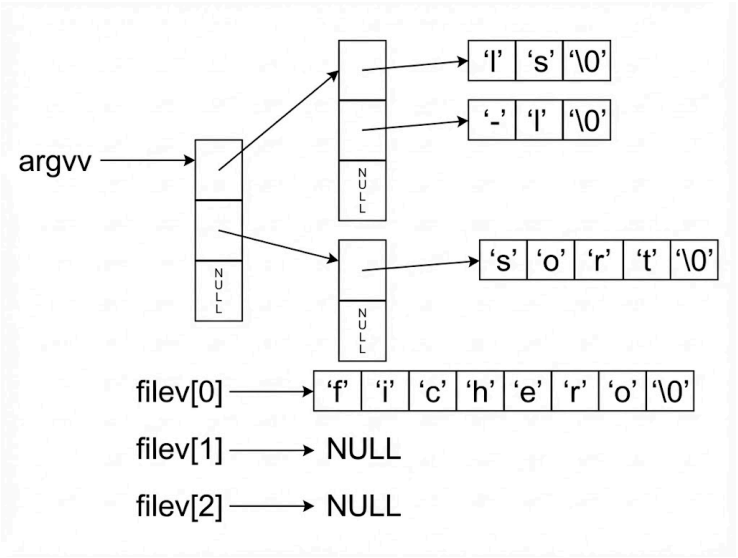

Figura 1: Estructura de datos generada por Uc3mshell.

Nota: Para familiarizarse con las estructuras en las que se devuelven los comandos, las redirecciones y el background, se recomienda al alumno compilar y ejecutar el programa proporcionado para probar diferentes comandos y secuencias de comandos en el fichero de entrada. El objetivo es comprender claramente cómo acceder a cada uno de los comandos y asegurarse de que se decodifiquen correctamente. No obstante, para ello, el alumno debe implementar la lectura línea por línea del fichero de entrada y hacer uso de la función facilitada: *int procesar_linea(char *linea);*

4. Desarrollo del intérprete de comandos Uc3mshell

Para desarrollar el Uc3mshell se recomienda al alumno seguir una serie de pasos, de tal forma que se construya el uc3mshell de forma incremental. En cada paso se añadirá una nueva funcionalidad sobre la anterior.

1. Lectura del fichero de entrada, carácter por carácter, reconocimiento de las líneas de comandos y añadido de '\0' como delimitador.
2. Ejecución de comandos simples como ls, who, etc.
3. Ejecución de comandos simples en "background" (&).
4. Ejecución de secuencias de comandos conectados por pipes (|). El número de comandos en una secuencia se limitará a 3, por ejemplo: ls –l | sort | wc. Si los alumnos deciden implementar una secuencia ilimitada de pipes **se considerará para nota complementaria** (nota máxima 10).
5. Ejecución de comandos simples y secuencias de comandos con redirecciones de entrada, de salida estándar y de error (<, >, !>) y en background (&).
6. Ejecución de comandos internos y externos, los cuales se detallan a continuación.

4.1. Comando interno: exit

El comando interno exit debe liberar la memoria utilizada, si la hubiese, esperar la finalización de los procesos ejecutados en segundo plano, eliminar procesos zombies, imprimir el mensaje de despedida "**Goodbye <número entero>**" y finalizar la ejecución del Uc3mshell mediante la llamada al sistema exit(). Recibe como argumento un número entero que debe imprimirse antes de terminar. Por ejemplo:

```shell
echo "Hola mundo"
exit pepe
exit
exit 3

# Salida esperada
Hola mundo
[ERROR] El código de salida debe ser un número entero
[ERROR] Falta código de salida
Goodbye 3
```

4.2. Comando interno: mycalc

El Uc3mshell debe soportar el comando interno mycalc cuya sintaxis es:

mycalc <operando 1> < + | - | x | / > <operando 2>

Para ello debe:

- Comprobar que la sintaxis es correcta.
- Ejecutar la operación elegida: + (suma), - (resta), x (multiplicación), / (división).
- Mostrar por pantalla los resultados.

Por ejemplo:

```shell
echo "Hola mundo"
mycalc 1 + 1

# Salida esperada
Hola mundo
Operación: 1 + 1 = 2
```

Al tratarse de un comando interno, se deberá crear un fichero de cabecera (mycalc.h) donde se declararán todas las funciones necesarias para invocar el comando *mycalc*, y se definirá su comportamiento en un fichero de implementación (mycalc.c). En el fichero fuente del Uc3mshell (uc3mshell.c) se deberá incluir el fichero de cabecera creado (#include "mycalc.h"), y **la invocación a *mycalc* se realizará mediante llamada a función**.

4.3. Comando externo: mycp

El comando *mycp* es un programa externo que debe ser desarrollado y compilado por el alumno de forma independiente al Uc3mshell, es decir, debe generarse un ejecutable al compilarlo. El funcionamiento de este programa es similar al del comando "**cp**" básico implementado en la shell de Linux: debe copiar un fichero, creando una copia. El nombre de la copia se da como parámetro.

El Uc3mshell debe ser capaz de ejecutar *mycp* **como si un comando nativo de Linux se tratase,** cuya sintaxis es:

mycp <fichero origen> <fichero destino>

Para ello debe:

- Comprobar que la sintaxis es correcta.
- Comprobar que el fichero de origen existe y leerlo.
- Crear un nuevo fichero con el nombre de destino especificado y escribir en él.
- Si el fichero de destino existe, este debe ser sobreescrito.

- Cerrar los ficheros abiertos.
- *Nota*: verificar que el nuevo fichero existe.

Por ejemplo:

```shell
echo "Hola mundo"
mycp file1.txt renamed.txt

# Salida esperada
Hola mundo
```

5. Código fuente de apoyo

Para facilitar la realización de esta práctica, se dispone del fichero ***p2_uc3mshell.zip*** que contiene código fuente de apoyo y se puede descargar de Aula Global. Para extraer su contenido se deberá ejecutar lo siguiente:

```shell
unzip p2_uc3mshell.zip
```

Al extraer su contenido, se crea el directorio **p2_uc3mshell/**, donde debe desarrollarse la práctica. Dentro de este directorio se incluyen los siguientes ficheros:

Makefile

Fichero fuente para la herramienta make. **No se debe modificar.** Con él se consigue la compilación automática solo de los ficheros fuente que se modifiquen.

uc3mshell.c

Fichero fuente de C. Este fichero es el que debe modificarse para el desarrollo del procesador de comandos.

mycp.c

Fichero fuente de C. Este fichero es el que debe modificarse para el desarrollo de mycp.

mycalc.c

Fichero fuente de C. Este fichero es el que debe modificarse para la definición de las funciones de mycalc.

mycalc.h

Fichero de cabeceras de C. Este fichero es el que debe modificarse para la declaración de las funciones de mycalc.

autores.txt

Fichero en formato de texto (.txt) donde se deben incluir los autores de la práctica (uno por línea). Formato (sin tildes):
NIA1,Apellido(s),Nombre(s)
NIA2,Apellido(s),Nombre(s)
NIA3,Apellido(s),Nombre(s)

```Textproto
Ejemplo válido:
100499455,Apellido1 Apellido2,Nombre
100499455,Apellido1 Apellido2,Nombre Compuesto
```

```Textproto
Ejemplo NO válido:
,Apellido1 Apellido2,Nombre1 (no incluye el NIA)
(línea en blanco)
Apellido1 Apellido2,Nombre Compuesto,100499455
499455,Apellido1 Apellido2,Nombre (NIA incompleto)
```

5.1. Comprobador proporcionado

Se proporciona a los estudiantes un script de Python (versión 3) llamado "**comprobador_ssoo_p2.py**" que verifica que el formato de la tarea de laboratorio sea correcto. El script comprueba que la tarea cumpla con las especificaciones de nomenclatura, esté correctamente comprimida, incluya todo lo especificado y se compile correctamente.

Para ejecutar el comprobador se deberá utilizar el siguiente comando:

```Shell
python3 comprobador_ssoo_p2.py <entregable.zip>
```

Siendo **entregable.zip** el fichero que se va a entregar por Aula Global (ver el apartado de Entrega). Por ejemplo:

```Shell
python3 comprobador_ssoo_p2.py ssoo_p2_100254896_100047014.zip
```

El comprobador imprimirá mensajes por pantalla indicando si el formato es o no correcto.

6. Compilación y ejecución

Para poder compilar y ejecutar el laboratorio, es necesario utilizar el comando “make”. Las reglas iniciales se encuentran ya en el fichero Makefile proporcionado.

Ejemplo de **compilación**:

```shell
make clean
make uc3mshell
make mycp
```

Ejemplo de **ejecución**:

```shell
./uc3mshell <fichero de entrada>
```

7. Entrega

Las prácticas se realizarán en equipos de **máximo 3 integrantes** pertenecientes al **mismo grupo reducido**..

7.1. Plazo de entrega

La fecha límite de entrega de la práctica en AULA GLOBAL será el **viernes 10 de abril de 2026 (hasta las 23:55h).**

7.2. Procedimiento de entrega de las prácticas

La entrega de las prácticas ha de realizarse de forma electrónica. En AULA GLOBAL se habilitarán enlaces a través de los cuales podrá entregar las prácticas. En concreto, **se habilitará un entregador para el código de la práctica y otro de tipo TURNITIN para la memoria de la práctica**.

7.3. Documentación a entregar

En el entregador para el código se debe entregar un archivo comprimido en formato zip con el nombre:

ssoo_p2_NIA1_NIA2_NIA3.zip

donde NIA1, NIA2 y NIA3 son los NIAs de los integrantes del grupo. El archivo debe contener:

- uc3mshell.c
- Makefile
- mycp.c
- mycalc.c
- mycalc.h
- autores.txt: **Fichero en formato de texto (.txt) donde se deben incluir los autores de la práctica (uno por línea). Formato (sin tildes) y como se ha indicado anteriormente.**

En el entregador TURNITIN deberá entregarse la memoria de la práctica en formato PDF con el nombre:

ssoo_p2_NIA1_NIA2_NIA3.pdf

La memoria tendrá que contener al menos los siguientes apartados:

- **Portada:** con los nombres completos de los autores, NIAs y direcciones de correo electrónico.
- **Índice.**
- **Descripción del código** detallando las principales funciones implementadas. NO incluir código fuente de la práctica en este apartado. Cualquier código será automáticamente ignorado.
- **Batería de pruebas** utilizadas y resultados obtenidos. Se dará mayor puntuación a pruebas avanzadas, casos extremos y, en general, a aquellas pruebas que garanticen el correcto funcionamiento de la práctica en todos los casos. Hay que tener en cuenta:
    - Que un programa compile correctamente y sin advertencias (*warnings*) no es garantía de que funcione correctamente.
    - Evite pruebas duplicadas que evalúan los mismos flujos de programa. La puntuación de este apartado no se mide en función del número de pruebas, sino del grado de cobertura de las mismas. Es mejor pocas pruebas que evalúan diferentes casos que muchas pruebas que evalúan siempre el mismo caso.
- **Conclusiones**, problemas encontrados, cómo se han solucionado y opiniones personales.

Se puntuará también los siguientes aspectos relativos a la **presentación** de la práctica:

- Debe contener portada, con los autores de la práctica y sus NIAs.
- Debe contener índice de contenidos.

- La memoria debe tener números de página en todas las páginas (menos la portada).
- El texto de la memoria debe estar justificado.

La longitud de la memoria no deberá superar las 15 páginas (portada e índice incluidos). Es imprescindible aprobar la memoria para aprobar la práctica, por lo que no debe descuidar la calidad de la misma.

Nota: es posible entregar el código de la práctica tantas veces como se quiera dentro del plazo de entrega, siendo la última entrega realizada la versión definitiva.
Consideraciones del uso de IA: Si se usan herramientas de IA y las soluciones de varios grupos coinciden, se considerará copia. También deben considerar que en este curso se incluye un examen parcial y un examen final, que requiere que los estudiantes escriban programas en C. Si no se tiene dominio suficiente, no podrán completar estos exámenes correctamente.
LA MEMORIA DE LA PRÁCTICA ÚNICAMENTE SE PODRÁ ENTREGAR UNA ÚNICA VEZ A TRAVÉS DE TURNITIN

8. Normas

1. Las prácticas que no compilen o que no se ajusten a la funcionalidad y requisitos planteados obtendrán una calificación de 0.

2. Los programas que utilicen funciones de biblioteca (**fopen, fread, fwrite, fclose, etc.**) o similares, en vez de llamadas al sistema, obtendrán una calificación de 0. Tampoco se permite utilizar sentencias o funciones como **goto**.

3. Se prestará especial atención a detectar funcionalidades copiadas entre dos prácticas. En caso de encontrar implementaciones comunes en dos prácticas, los alumnos involucrados (copiados y copiadores) perderán las calificaciones obtenidas por evaluación continua.

4. Los programas deben compilar sin **warnings**.

5. Los programas deberán funcionar bajo un sistema Linux, no se permite la realización de la práctica para otros sistemas (como Windows o macOS). Además, para asegurarse del correcto funcionamiento de la práctica, deberá chequearse su compilación y ejecución en el servidor Guernika (https://www.lab.inf.uc3m.es/servicios/aulas-virtuales-del-laboratorio/). Si el código presentado no compila o no funciona sobre esta plataforma, la implementación no se considerará correcta.

6. Un programa no comentado obtendrá una **calificación muy baja**.

7. La entrega de la práctica se realizará a través de Aula Global, tal y como se detalla en el apartado Entrega de este documento. No se permite la entrega a través de correo electrónico sin autorización previa.

8. Se debe respetar en todo momento el formato de la entrada y salida que se indica en cada programa a implementar.

9. Se debe realizar un control de errores en cada uno de los programas, **más exhaustivo de lo solicitado explícitamente en cada apartado**.

Los programas entregados que no sigan estas normas no se considerarán aprobados.

9. Anexos

9.1. man function

man es el paginador del manual del sistema, es decir permite buscar información sobre un programa, una utilidad o una función. Véase el siguiente ejemplo:

man 2 fork

Las páginas usadas como argumentos al ejecutar man suelen ser, por lo general, nombres de programas, utilidades o funciones. Normalmente, la búsqueda se lleva a cabo en todas las secciones de manual disponibles según un orden predeterminado, y sólo se presenta la primera página encontrada, incluso si esa página se encuentra en varias secciones.

Para salir de la página mostrada, basta con pulsar la tecla 'q'.

Una página de manual tiene varias partes. Estas están etiquetadas como NOMBRE, SINOPSIS, DESCRIPCIÓN, OPCIONES, FICHEROS, VÉASE TAMBIÉN, BUGS, y AUTOR. En la etiqueta de SINOPSIS se recogen las librerías (identificadas por la directiva *#include*) que se deben incluir en el programa en C del usuario para poder hacer uso de las funciones correspondientes.

Las formas más comunes de usar *man* son las siguientes:

- **man sección elemento:** Presenta la página de elemento disponible en la sección del manual.
- **man -a elemento:** Presenta, secuencialmente, todas las páginas de los elementos disponibles en el manual. Entre página y página se puede decidir saltar a la siguiente o salir del paginador completamente.
- **man -k palabra-clave:** Busca la palabra-clave entre las descripciones breves y las páginas de manual y presenta todas las que casen.

9.2. Modo background y foreground

Cuando una secuencia de comandos se ejecuta en background, el *pid* que se imprime es el del proceso que ejecuta el último mandato de la secuencia.

Cuando un mandato simple se ejecuta en background, el *pid* que se imprime es el del proceso que ejecuta este mandato.

Con la operación de background, es posible que el proceso uc3mshell muestre el prompt entremezclado con la salida del proceso hijo. Esto es correcto.

Después de ejecutar un mandato en foreground, la uc3mshell no puede tener procesos zombies de comandos anteriores ejecutados en background.

9.3. Comandos externos

Los comandos externos (mycp) se ejecutan como un programa cualquiera, ejecutado sobre un proceso. Por lo tanto:

- Forman parte de secuencias de comandos,
- Tienen redirecciones de ficheros,
- Se ejecutan en background.

10. Bibliografía

- El lenguaje de programación C: diseño e implementación de programas Félix García, Jesús Carretero, Javier Fernández y Alejandro Calderón. Prentice-Hall, 2002.
- The UNIX System S.R. Bourne Addison-Wesley, 1983.
- Advanced UNIX Programming M.J. Rochkind Prentice-Hall, 1985.
- Sistemas Operativos: Una visión aplicada. Jesús Carretero, Félix García, Pedro de Miguel y Fernando Pérez. McGraw-Hill, 2001.
- Unix man pages (man function)